

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования «Тульский государственный университет»
Институт прикладной математики и компьютерных наук

ЛАБОРАТОРНАЯ РАБОТА №9
по дисциплине
Методы и средства защиты информации

по теме
«Защита электронных документов с использованием цифровых водяных
знаков»

Выполнил: ст. гр. 220032-11

Лаукерт К. М.

Принял: к.т.н., доцент ИПМКН

Кулешова Н. В.

Тула 2026

Лабораторная работа №9

по теме «Защита электронных документов с использованием цифровых водяных знаков»

Цель работы

Освоить методы внедрения цифровых водяных знаков в графические файлы для защиты авторских прав, контроля распространения контента и предотвращения несанкционированного использования.

Задание

Создать видимый текстовый водяной знак, экспортировать изображение в JPEG, PNG, GIF, BMP и TIFF, затем проверить устойчивость знака к преобразованиям из методических указаний.

Использованный метод

Реализована локальная программа watermark_lab.py на Python с библиотекой Pillow. Она создаёт учебное изображение-контейнер, накладывает текст «© МиСЗКИ • ЛАУКЕРТ К. М.» белым полужирным шрифтом по диагонали и задаёт прозрачность 40%. Такой видимый ЦВЗ позволяет сразу установить автора, однако не является стойким криптографическим доказательством авторства.

Создание водяного знака



Рисунок 1 — Исходное учебное изображение и результат нанесения диагонального ЦВЗ с прозрачностью 40%

Сформированы файлы с одним и тем же видимым знаком в форматах JPEG, PNG, GIF, BMP и TIFF. При сохранении в GIF палитра квантуется, в JPEG применяются потери сжатия; PNG,

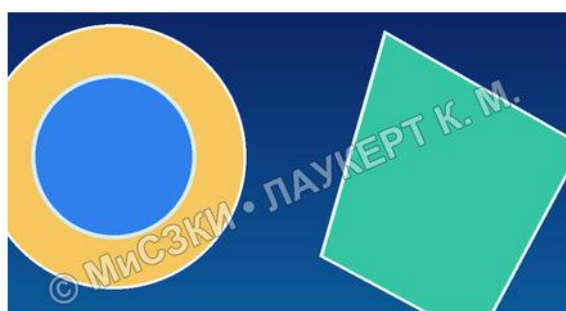
BMP и TIFF сохраняют качество исходного растрового представления.



Сжатие JPEG, качество 60%



Сжатие JPEG, качество 80%



Обрезка 10% краёв



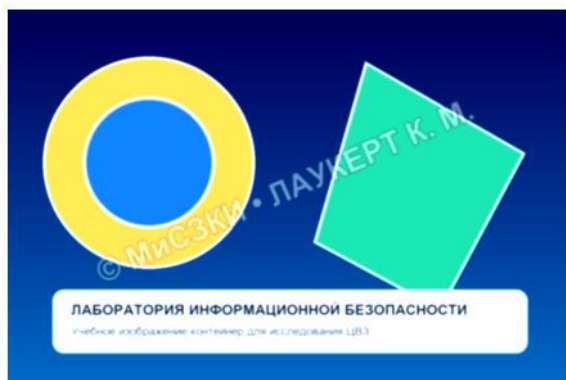
Поворот +15° и возврат



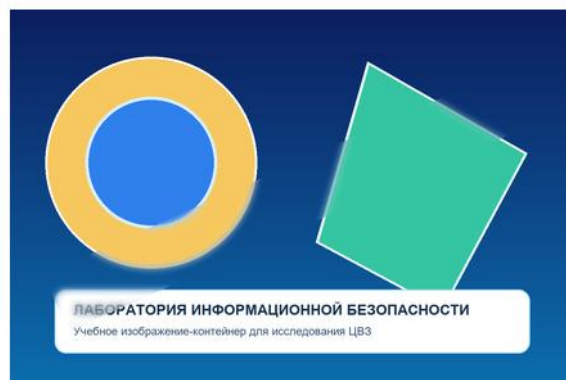
Уменьшение до 50%



Увеличение до 150%



Размытие и контраст



Удаление (ретушь) ЦВЗ

Рисунок 2 — Результаты преобразований изображения в формате PNG

Результаты тестирования устойчивости

№	Воздействие	Результат для JPEG / PNG / GIF / BMP / TIFF
1	Сжатие с качеством 60%	Знак остаётся читаемым во всех форматах. В JPEG появляются артефакты, но контур текста сохраняется.
2	Сжатие с качеством 80%	Знак читаем; визуальные изменения минимальны.
3	Обрезка 10% краёв	Центральный диагональный знак сохраняется, но его части у краёв теряются.
4	Поворот на 15° и возврат	Знак читаем, однако изображение получает интерполяционные искажения и пустые области по углам.
5	Уменьшение до 50%	Знак остаётся читаемым, но мелкие детали шрифта сглаживаются.
6	Увеличение до 150%	Знак читаем; появляются размытость и пикселизация.
7	Размытие и контраст	Знак сохраняется, его контраст меняется; размытие снижает чёткость границ.
8	Удаление ЦВЗ	При ретуши знак исчезает, но обычно остаются следы: потеря деталей, неоднородность текстуры или повторяющиеся фрагменты фона.

Сравнение форматов

PNG, BMP и TIFF сохранили наиболее чистый вид знака, поскольку не используют потерь при кодировании растра. TIFF удобен для архивного хранения, но имеет большой размер. JPEG лучше подходит для публикации фотографий, но чувствителен к повторному сжатию. GIF применяет ограниченную палитру, поэтому на градиентах заметны ступенчатые переходы. При рассмотренных операциях сам факт видимого ЦВЗ определяется прежде всего его расположением и контрастом, а не форматом контейнера.

Результат выполнения

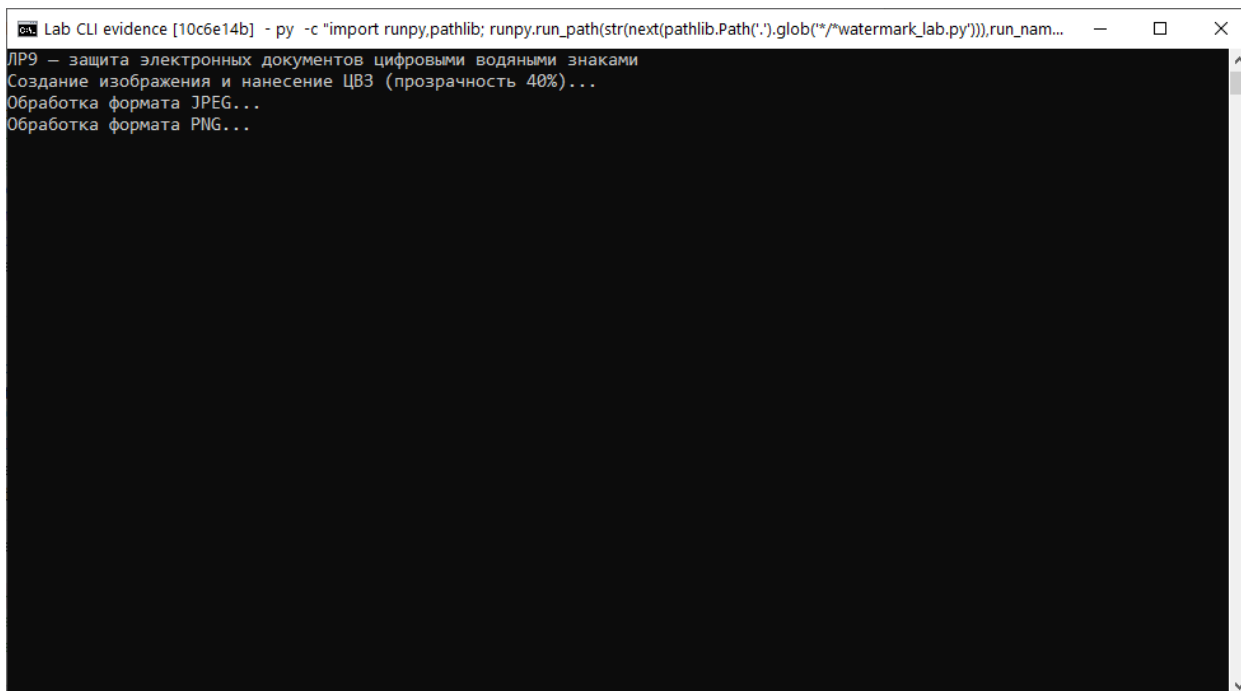


Рисунок 3 — Фактический запуск программы: создание форматов и выполнение тестов устойчивости

Листинг программы

```
"""ЛР9: создание видимого цифрового водяного знака и тест
устойчивости.

Требуется Pillow: py -m pip install Pillow.

"""

from __future__ import annotations

from pathlib import Path

from PIL import Image, ImageDraw, ImageEnhance, ImageFilter,
ImageFont

ROOT = Path(__file__).resolve().parent
OUT = ROOT / "output"

FORMATS = {"JPEG": "jpg", "PNG": "png", "GIF": "gif", "BMP":
"bmp", "TIFF": "tiff"}

ACTIONS = [
    ("01_quality_60", "Сжатие JPEG, качество 60%"),
    ("02_quality_80", "Сжатие JPEG, качество 80%"),
    ("03_crop_10", "Обрезка 10% краёв"),
    ("04_rotate_restore", "Поворот +15° и возврат"),
    ("05_reduce_50", "Уменьшение до 50%"),
    ("06_enlarge_150", "Увеличение до 150%"),
```

```

        ("07_blur_contrast", "Размытие и контраст"),
        ("08_removed", "Удаление (пелушь) ЦВЗ"),
    ]

def font(size: int, bold: bool = False):
    names = ["DejaVuSans-Bold.ttf", "arialbd.ttf"] if bold else
    ["DejaVuSans.ttf", "arial.ttf"]
    for name in names:
        try:
            return ImageFont.truetype(name, size)
        except OSError:
            pass
    return ImageFont.load_default()

def make_original() -> Image.Image:
    """Создаёт самостоятельное учебное изображение-контейнер."""
    w, h = 1200, 800
    img = Image.new("RGB", (w, h), "#102a43")
    d = ImageDraw.Draw(img)
    for y in range(h):
        c = int(30 + 80 * y / h)
        d.line((0, y, w, y), fill=(12, c, min(180, c + 65)))
        d.ellipse((75, 100, 525, 550), fill="#f6c85f",
        outline="#ffffff", width=5)
        d.ellipse((160, 185, 440, 465), fill="#2f80ed",
        outline="#d9efff", width=7)
        d.polygon([(760, 110), (1100, 305), (920, 640), (650, 495)],
        fill="#36c5a2", outline="#e9fffa", width=6)
        d.rounded_rectangle((95, 595, 1105, 730), radius=24,
        fill="#ffffff", outline="#b9e4ff", width=3)
        d.text((135, 620), "ЛАБОРАТОРИЯ ИНФОРМАЦИОННОЙ
        БЕЗОПАСНОСТИ", fill="#16324f", font=font(31, True))
        d.text((135, 670), "Учебное изображение-контейнер для
        исследования ЦВЗ", fill="#31506e", font=font(24))
    return img

```

```

def apply_watermark(image: Image.Image) -> Image.Image:
    """Наносит видимый диагональный текст с прозрачностью
    40%."""
    layer = Image.new("RGBA", image.size, (0, 0, 0, 0))
    mark = Image.new("RGBA", (1100, 180), (0, 0, 0, 0))
    md = ImageDraw.Draw(mark)
    text = "© МиСЗКИ • ЛАУКЕПТ К. М."
    f = font(66, True)
    bbox = md.textbbox((0, 0), text, font=f, stroke_width=2)
    x = (1100 - (bbox[2] - bbox[0])) // 2
    md.text((x, 35), text, font=f, fill=(255, 255, 255, 102),
    stroke_width=2, stroke_fill=(12, 42, 67, 102))
    mark = mark.rotate(24, expand=True,
    resample=Image.Resampling.BICUBIC)
    layer.alpha_composite(mark, ((image.width - mark.width) //
    2, (image.height - mark.height) // 2))
    return Image.alpha_composite(image.convert("RGBA"),
    layer).convert("RGB")

```

```

def save(image: Image.Image, path: Path, fmt: str) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    if fmt == "JPEG":
        image.convert("RGB").save(path, fmt, quality=95,
        optimize=True)
    elif fmt == "GIF":
        image.convert("P",
        palette=Image.Palette.ADAPTIVE).save(path, fmt)
    else:
        image.convert("RGB").save(path, fmt)

```

```

def jpeg_roundtrip(image: Image.Image, quality: int) ->
Image.Image:
    temp = OUT / "temporary.jpg"
    image.save(temp, "JPEG", quality=quality)
    result = Image.open(temp).convert("RGB").copy()

```

```

temp.unlink()
return result

def transform(image: Image.Image, action: str, original:
Image.Image) -> Image.Image:
    w, h = image.size
    if action == "01_quality_60": return jpeg_roundtrip(image,
60)
    if action == "02_quality_80": return jpeg_roundtrip(image,
80)
    if action == "03_crop_10":
        cropped = image.crop((int(w*.1), int(h*.1), int(w*.9),
int(h*.9)))
        return cropped.resize((w, h), Image.Resampling.LANCZOS)
    if action == "04_rotate_restore":
        # Двойной поворот сохраняет итоговый размер, но
добавляет заметные
        # интерполяционные искажения и области заполнения по
углам.
        rotated = image.rotate(15,
resample=Image.Resampling.BICUBIC, fillcolor="#1676b2")
        return rotated.rotate(-15,
resample=Image.Resampling.BICUBIC, fillcolor="#1676b2")
    if action == "05_reduce_50": return image.resize((w//2,
h//2), Image.Resampling.LANCZOS)
    if action == "06_enlarge_150":
        # Оставляем итоговый холст исходного размера: увеличение
до 150%
        # и центральная обрезка наглядно показывают масштаб и
пикселизацию.
        enlarged = image.resize((int(w * 1.5), int(h * 1.5)),
Image.Resampling.NEAREST)
        left = (enlarged.width - w) // 2
        top = (enlarged.height - h) // 2
        return enlarged.crop((left, top, left + w, top + h))
    if action == "07_blur_contrast": return
ImageEnhance.Contrast(image.filter(ImageFilter.GaussianBlur(2)))
.enhance(1.45)
    if action == "08_removed":

```

```

        # Имитация ретуши: текст удалён, но в его области
        # остаётся сглаженная полоса.
        mask = Image.new("L", (w, h), 0)
        band = Image.new("L", (1100, 180), 0)
        ImageDraw.Draw(band).rectangle((0, 25, 1100, 155),
        fill=185)
        band = band.rotate(24, expand=True,
        resample=Image.Resampling.BICUBIC)
        mask.paste(band, ((w - band.width) // 2, (h -
        band.height) // 2))
        return
        Image.composite(original.filter(ImageFilter.GaussianBlur(7)),
        original, mask)
        raise ValueError(action)

def sheet(items: list[tuple[str, Image.Image]], path: Path,
columns: int = 2) -> None:
    thumb_w, thumb_h = 440, 293
    rows = (len(items) + columns - 1) // columns
    page = Image.new("RGB", (columns * 470 + 30, rows * 350 +
    30), "white")
    d = ImageDraw.Draw(page)
    for index, (title, image) in enumerate(items):
        x = 20 + (index % columns) * 470
        y = 20 + (index // columns) * 350
        copy = image.copy(); copy.thumbnail((thumb_w, thumb_h),
        Image.Resampling.LANCZOS)
        page.paste(copy, (x, y))
        d.text((x, y + 298), title, fill="#102a43",
        font=font(18, True))
    page.save(path, "PNG")

def main() -> None:
    print("ЛР9 – защита электронных документов цифровыми
    водяными знаками", flush=True)
    print("Создание изображения и нанесение ЦВЗ (прозрачность
    40%)...", flush=True)
    OUT.mkdir(exist_ok=True)

```

```

original = make_original()
marked = apply_watermark(original)
save(original, OUT / "original.png", "PNG")
formats_dir = OUT / "formats"

for fmt, ext in FORMATS.items(): save(marked, formats_dir /
f"watermarked.{ext}", fmt)

sheet([("Оригинал", original), ("Видимый ЦВЗ: 40%,
диагональ", marked)], OUT / "watermark_steps.png")


results: dict[str, list[tuple[str, Image.Image]]] = {}
for fmt, ext in FORMATS.items():
    print(f"Обработка формата {fmt}...", flush=True)
    base = Image.open(formats_dir /
f"watermarked.{ext}").convert("RGB")
    result_set = []
    for action, title in ACTIONS:
        outcome = transform(base, action, original)
        save(outcome, OUT / "tests" / action /
f"{fmt.lower()}.{ext}", fmt)
        result_set.append((title, outcome))
    results[fmt] = result_set
    sheet(result_set, OUT / f"tests_{fmt.lower()}.png",
columns=2)


print("Создано: original.png и 5 форматов с видимым ЦВЗ
(прозрачность 40%).")
print("Проверка устойчивости (для всех форматов):")
for number, (_, title) in enumerate(ACTIONS, 1):
    verdict = "знак читаем" if number < 8 else "знак удалён,
остались признаки ретуши/потери деталей"
    print(f" {number}. {title}: {verdict}")

print("Результаты: output/formats, output/tests и обзорные
PNG-файлы.")


if __name__ == "__main__":
    main()

```

Ответы на контрольные вопросы

1. Цифровой водяной знак — данные, встроенные в цифровой объект для идентификации автора, контроля целостности или распространения.
2. Видимый знак воспринимается человеком как текст или логотип на изображении. Невидимый знак скрыт в пикселях или коэффициентах преобразования и обнаруживается специальным алгоритмом.
3. На устойчивость влияют способ встраивания, амплитуда изменения контейнера, выбор области, избыточность, ключ и характер атак: сжатие, масштабирование, обрезка, поворот и фильтрация.
4. По устойчивости различают робастные, хрупкие и полухрупкие ЦВЗ.
5. Невидимый знак сложнее удалить, потому что его расположение и ключ неизвестны, а изменение множества распределённых коэффициентов может существенно ухудшить контейнер.
6. Применения: защита авторских прав на медиаконтент, отслеживание распространения копий, контроль целостности электронных документов.

Вывод

Создан и протестирован видимый цифровой водяной знак. Он сохраняет читаемость после сжатия, масштабирования, поворота, фильтрации и умеренной обрезки, но может быть удалён ретушью. Поэтому видимый ЦВЗ служит удобным средством предупреждения и идентификации автора; для надёжного контроля авторства требуются устойчивые невидимые методы, например встраивание в частотные коэффициенты.

Список литературы

1. Баранов А. В., Киреев В. Ю. Стеганография и цифровые водяные знаки. М.: Горячая линия – Телеком, 2010.
2. Методические указания к лабораторной работе 9 «Защита электронных документов с использованием цифровых водяных знаков».